

Tema 4

Pregunta 1 (Definir el concepto de autómata con pila, i explicar quan una paraula és reconeguda per un autómata amb pila). Un autómata amb pila és una estructura $M = (K, \Sigma, \Gamma, \Delta, q_o, F)$ on:

1. K és un conjunt finit i no buit d'estats,
2. Σ és l'alfabet d'entrada,
3. Γ és l'alfabet de la pila,
4. $q_o \in K$ és l'estat inicial,
5. $F \subseteq K$ és el conjunt d'estats acceptadors,
6. Δ és un conjunt finit d'elements de la forma $((p, a, b), (q, x))$ amb $p, q \in K, a \in \Sigma \cup \{\lambda\}, b \in \Gamma \cup \{\lambda\}$ i $x \in \Gamma^*$.

Sigui $M = (K, \Sigma, \Gamma, \Delta, q_o, F)$ un autómata amb pila. Diem que una paraula $x \in \Sigma^*$ és reconeguda per M si existeix un còmput per a l'entrada x que acaba en un estat acceptador, llegeix tota la paraula d'entrada i deixa la pila buida al final del còmput.

Pregunta 2 (Definir el concepte d'autómata amb pila determinista). Sigui $M = (K, \Sigma, \Gamma, \Delta, q_o, F)$ un autómata amb pila. Dues transicions $((p_1, a_1, b_1), (q_1, \alpha_1))$ i $((p_2, a_2, b_2), (q_2, \alpha_2))$ de M són compatibles si:

1. $p_1 = p_2$,
2. $a_1 = a_2$ o $a_1 = \lambda$ o $a_2 = \lambda$,
3. $b_1 = b_2$ o $b_1 = \lambda$ o $b_2 = \lambda$.

Que dues transicions siguin compatibles significa que les dues es poden aplicar en un mateix pas de còmput de l'autómata. Diem que M és determinista si no té dues transicions compatibles diferents.

Pregunta 3 (Definir el concepte de gramàtica incontextual i explicar la relació amb els autòmats amb pila). Una gramàtica incontextual és una estructura $G = (V, \Sigma, P, S)$ on:

1. V és un alfabet, els seus elements s'anomenen variables,
2. Σ és un alfabet disjunt de V , els seus elements s'anomenen terminals,
3. P és un subconjunt finit de $V \times (V \cup \Sigma)^*$, els seus elements s'anomenen produccions (o regles),
4. $S \in V$ és la variable inicial.

Els autòmats amb pila són estructures equivalents a les gramàtiques incontextuals: per a tot llenguatge L , existeix un autómata amb pila M tal que $L = L(M)$ si i només si existeix una gramàtica incontextual G tal que $L = L(G)$.

Pregunta 4 (Definir el concepte de derivació en una gramàtica incontextual). Sigui $G = (V, \Sigma, P, S)$ una gramàtica incontextual i $u, v \in (V \cup \Sigma)^*$. Hi ha una derivació d'un pas de la paraula u a la paraula v , en símbols

$u \Rightarrow_G v$, si obtenim v a partir de u aplicant una producció de P . Hi ha una derivació de u a v , en símbols $u \Rightarrow_G^* v$, si obtenim v a partir de u aplicant un nombre finit de vegades les produccions de P .

Definim $L(G) = \{x \in \Sigma^* : S \Rightarrow_G^* x\}$.

Pregunta 5 (Mostrar l'algorisme vist a classe per construir un autòmat amb pila equivalent a una gramàtica incontextual). Si $G = (V, \Sigma, P, S)$ és una gramàtica incontextual, definim l'autòmat amb pila $M = (\{q_o, f\}, \Sigma, V \cup \Sigma, \Delta, q_o, \{f\})$, on Δ consta de les següents transicions:

1. $((q_o, \lambda, \lambda), (f, S))$.
2. $((f, \lambda, A), (f, x))$ per a cada producció $A \rightarrow x$ de P .
3. $((f, a, a), (f, \lambda))$ per a cada símbol terminal a de Σ .

Es pot demostrar, doncs, que $L(G) = L(M)$, és a dir, G i M són equivalents.

Pregunta 6 (Definir els conceptes d'arbre de derivació i gramàtica ambigua, i explicar per què les gramàtiques ambigües no s'han d'utilitzar en el disseny de compiladors). Sigui $G = (V, \Sigma, P, S)$ una gramàtica incontextual.

1. A tota derivació $S \Rightarrow_G^* x$ en G li associem el següent arbre:
 - La arrel de l'arbre és S .
 - Cada node intern correspon a una variable que es substitueix en el procés de derivació.
 - La paraula formada per les fulles, d'esquerra a dreta, és x .
2. L'arbre així obtingut s'anomena arbre de derivació de x .
3. La gramàtica G és ambigua si existeix una paraula $x \in L(G)$ amb més d'un arbre de derivació associat.

Les gramàtiques ambigües no s'han d'utilitzar en el disseny de compiladors perquè poden donar lloc a interpretacions diferents d'una mateixa instrucció, produint resultats d'execució diferents.

Pregunta 7 (Demostrar que és ambigua la gramàtica incontextual definida per les produccions:

$I \rightarrow \underline{if}(E) I \underline{else} I$

$I \rightarrow \underline{if}(E) I$). La gramàtica és ambigua perquè una instrucció de la forma $\underline{if}(E_1) \underline{if}(E_2) S_1 \underline{else} S_2$ (és a dir, dos ifs i un sol else) té dos arbres de derivació diferents, corresponents a associar l'else al primer o al segon if.

Pregunta 8 (Explicar en què consisteixen les tres fases del disseny d'un compilador i en quines fases s'utilitzen els autòmats deterministes, els autòmats amb pila i les gramàtiques incontextuals). El disseny d'un compilador consta de les següents fases:

1. **Anàlisi lèxic:** Agrupa els símbols del programa font en categories sintàctiques (tokens) i passa aquesta informació a l'analitzador sintàctic. Aquí s'utilitzen autòmats deterministes.
2. **Anàlisi sintàctic:** Determina si el programa font segueix les regles del llenguatge i construeix l'arbre de derivació. Aquí s'utilitzen gramàtiques incontextuals i autòmats amb pila.

3. **Anàlisi semàntica:** Comprova la compatibilitat de tipus i altres aspectes semàntics, i si no hi ha errors, tradueix el programa a codi màquina. També s'utilitzen gramàtiques incontextuals i autòmats amb pila.

Pregunta 9 (Explicar l'interès que té el disseny de compiladors per a la programació). Sense compiladors no seria possible programar en llenguatges d'alt nivell, ja que un compilador transforma el programa font en un fitxer executable. A més, els compiladors permeten detectar i corregir errors en els programes i abordar problemes complexos que serien impossibles d'afrontar en llenguatge ensamblador.

Pregunta 10 (Mostrar l'algoritme vist a classe per dissenyar un analitzador sintàctic). L'algoritme té tres etapes:

1. Dissenyar una gramàtica incontextual no ambigua que generi el llenguatge.
2. Transformar la gramàtica en una altra equivalent tal que l'autòmat amb pila associat sigui determinista.
3. Programar l'autòmat amb pila associat a la gramàtica del pas anterior.

Pregunta 11 (Explicar com es construeix la taula d'anàlisi d'una gramàtica incontextual). Sigui $G = (V, \Sigma, P, S)$ una gramàtica incontextual.

1. Per a cada paraula $\alpha \in (V \cup \Sigma)^*$, definim el conjunt **Primeros**(α) com el conjunt de símbols terminals que poden aparèixer al començament d'una paraula derivada a partir de α . Si la paraula buida λ es deriva de α , també s'inclou λ a **Primeros**(α).
2. Per a cada variable $A \in V$, definim **Sigüientes**(A) com el conjunt de símbols terminals $a \in \Sigma$ tals que $S \Rightarrow_G^* \alpha A a \beta$ per a algunes $\alpha, \beta \in (V \cup \Sigma)^*$.
3. Si $A \in V$ i $a \in \Sigma$, definim $\text{TAULA}[A, a] = \{(A, \beta) \in P : a \in \text{Primeros}(\beta \cdot \text{Sigüientes}(A))\}$.

Pregunta 12 (Definir el concepte de gramàtica LL(1) i explicar l'interès de les gramàtiques LL(1) en el disseny de compiladors). Una gramàtica incontextual G és LL(1) si el procés de construcció de la taula d'anàlisi no genera conflictes, és a dir, si per a tot $A \in V$ i $a \in \Sigma$, $\text{TAULA}[A, a]$ no conté més d'un element.

Les gramàtiques LL(1) són interessants per al disseny de compiladors perquè permeten programar directament l'autòmat amb pila associat a la gramàtica utilitzant la taula d'anàlisi.

Pregunta 13 (Explicar l'algoritme vist a classe per programar l'autòmat amb pila associat a una gramàtica LL(1)). L'algorisme és el següent.

```

public boolean analisi_sintactic(String entrada) {
    Stack<Character> pila = new Stack<Character>();
    int q = 0, i = 0;
    boolean b = true;
    char c = entrada.charAt(0);
    // (1) Posar '$' a la pila
    // (2) Aplicar la transició ((q0, λ, λ), (f, S))
    while ((tope de la pila != '$' || caracter d'entrada != '$') && b) {
        if (tope de la pila == 'a' && caracter d'entrada == 'a') {
            // aplicar la transició ((f, a, a), (f, λ))
            // llegir següent caràcter
        } else if (tope de la pila == 'A' && caracter d'entrada == 'a'
            && TAULA[A, a] == A → X1...Xn) {
            // aplicar la transició ((f, λ, A), (f, X1...Xn))
        } else {
            b = false;
        }
    }
    if (tope de la pila == '$' && caracter d'entrada == '$') return true;
    else return false;
}

```

Pregunta 14 (Explicar en què consisteix la regla de factorització). La regla de factorització, que permet transformar moltes gramàtiques no ambigües en gramàtiques LL(1), afirma el següent:

Si tenim produccions de la forma $A \rightarrow \alpha\beta_1 \mid \dots \mid \alpha\beta_n$ amb $\alpha \neq \lambda$ i $n \geq 2$, es reemplacen per

$$A \rightarrow \alpha A', \quad A' \rightarrow \beta_1 \mid \dots \mid \beta_n$$

on A' és una nova variable.

Pregunta 15 (Explicar en què consisteix la regla de recursió). La regla de recursió, que permet transformar moltes gramàtiques no ambigües en gramàtiques LL(1), afirma el següent:

Si tenim produccions de la forma $A \rightarrow A\alpha_1 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \dots \mid \beta_m$ amb $n \geq 1$ i les β_i no comencen per A , es reemplacen per:

$$A \rightarrow \beta_1 A' \mid \dots \mid \beta_m A', \quad A' \rightarrow \alpha_1 A' \mid \dots \mid \alpha_n A' \mid \lambda$$

on A' és una nova variable.